

Fine-tuning

1. The big picture

When you fine-tune a model, the basic process is:

```
Dataset
  ↓
Split into batches
  ↓
Model sees a batch
  ↓
Forward pass
  ↓
Calculate loss
  ↓
Backward pass
  ↓
Calculate gradients
  ↓
Optimizer updates weights
  ↓
Repeat
  ↓
Eventually complete one epoch
  ↓
Start next epoch
```

The most important distinction is:

Batch = how much data the model sees at once

Iteration = one batch being processed

Step = usually one optimizer update

Epoch = one complete pass through the dataset

2. Dataset

Suppose you have **10,000 training examples**.

For an LLM, an example could be:

```
User:
Explain CAP theorem.

Assistant:
CAP theorem states that...
```

Your dataset might contain:

```
10,000 examples
```

But you normally don't feed all 10,000 examples into GPU memory at once.
Instead, you divide them into **batches**.

3. Batch

Suppose:

```
Dataset = 10,000 examples
Batch size = 100
```

The model processes:

```
Batch 1 → examples 1-100
Batch 2 → examples 101-200
Batch 3 → examples 201-300
...
Batch 100 → examples 9901-10000
```

So:

```
10,000 / 100 = 100 batches
```

Therefore, one complete pass through the dataset requires **100 batches**.

4. Epoch

An **epoch** means:

| The model has gone through the entire training dataset once.

For example:

```
Dataset = 10,000 examples  
Batch size = 100  
  
100 batches = 1 epoch
```

If you train for:

```
3 epochs
```

the model sees the dataset three times:

```
Epoch 1  
  Batch 1  
  Batch 2  
  ...  
  Batch 100  
  
Epoch 2  
  Batch 1  
  ...  
  Batch 100  
  
Epoch 3  
  Batch 1  
  ...  
  Batch 100
```

Total batches processed:

```
100 × 3 = 300
```

5. Iteration

An **iteration** generally means:

| One batch goes through the training process.

For example:

```
Iteration 1 → Batch 1  
Iteration 2 → Batch 2  
Iteration 3 → Batch 3  
...
```

So if:

```
10,000 examples  
batch size = 100
```

you have:

```
100 iterations per epoch
```

And with 3 epochs:

```
300 iterations
```

Important terminology warning

Different libraries sometimes use **iteration** and **step** slightly differently.

In basic training discussions:

```
1 iteration ≈ 1 batch processed
```

But when gradient accumulation is involved, you need to distinguish:

```
micro-step / iteration
  ↓
gradient accumulation
  ↓
optimizer step
```

This is extremely important for LLM fine-tuning.

6. Forward pass

Now let's take one batch.

The model receives:

```
Input
  ↓
Transformer
  ↓
Prediction
```

For example:

```
Input:
"The capital of Pakistan is"

Model prediction:
"Karachi"
```

But suppose the correct answer is:

```
"Islamabad"
```

The model's prediction isn't perfect.

We need to measure **how wrong it was**.

That's where loss comes in.

7. Loss

Loss = a numerical measurement of how wrong the model's prediction is.

For example:

```
Loss = 2.5
```

After training:

```
Loss = 1.2
```

Lower loss generally means the model is doing better on the training objective.

For language models, the usual training objective is based on **next-token prediction**.

Example:

```
The capital of Pakistan is ____
```

Correct next token:

```
Islamabad
```

The model gives probabilities:

Islamabad	0.70
Karachi	0.20
Lahore	0.05
Peshawar	0.05

The loss measures how well those probabilities align with the correct token.

8. Gradient

Now the model knows:

```
Loss = 2.5
```

But that's not enough.

We need to know:

| Which model parameters should change, and in which direction?

That's what gradients tell us.

Imagine the model has a parameter:

```
weight = 0.8
```

The gradient might tell us:

```
gradient = +0.3
```

This tells the optimizer roughly how changing that parameter affects the loss.

For millions/billions of parameters, you get gradients such as:

```
weight 1 → gradient  
weight 2 → gradient  
weight 3 → gradient  
...  
weight N → gradient
```

9. Backpropagation

The process of calculating those gradients is called:

| Backpropagation

Conceptually:

```
Input
↓
Forward pass
↓
Prediction
↓
Loss
↓
Backpropagation
↓
Gradients
```

So:

Forward pass → calculate prediction

Loss → calculate error

Backpropagation → calculate gradients

10. Optimizer

Now we have gradients.

We need something that actually changes the model's parameters.

That's the job of the **optimizer**.

Common optimizers:

```
SGD
Adam
AdamW
```

LLM fine-tuning commonly uses:

```
AdamW
```

Very simplified:

```
new_weight = old_weight - learning_rate × gradient
```

For example:

```
old weight = 0.80  
gradient = 0.30  
learning rate = 0.01
```

Approximately:

```
new weight = 0.80 - (0.01 × 0.30)  
  
            = 0.797
```

The optimizer performs this update across the trainable parameters.

11. Learning rate

The **learning rate (LR)** controls:

| How large each parameter update should be.

Example:

```
Learning rate = 0.001
```

Small LR:

```
small updates  
→ slower learning  
→ potentially more stable
```

Large LR:

```
large updates
→ faster movement
→ potentially unstable
```

For fine-tuning large language models, learning rates are usually much smaller than you might use when training a model from scratch.

For example, depending on the method:

```
1e-4
5e-5
2e-5
1e-5
```

But the appropriate value depends heavily on the model, dataset, LoRA/full fine-tuning, batch size, etc.

12. Optimizer step

This is where **step** becomes important.

An **optimizer step** means:

| The optimizer actually updates the model's parameters.

Conceptually:

```
Forward
↓
Loss
↓
Backward
↓
Gradients
↓
Optimizer.step()
↓
Weights updated
```

That last part is the **optimizer step**.

13. Gradient accumulation

Now we get to one of the most important concepts for LLM fine-tuning.

Suppose your GPU cannot handle:

```
batch size = 32
```

but it can handle:

```
batch size = 8
```

You could use:

```
batch size = 8  
gradient accumulation = 4
```

The model processes:

```
Batch 1 → calculate gradients  
Batch 2 → calculate gradients  
Batch 3 → calculate gradients  
Batch 4 → calculate gradients  
      ↓  
    accumulate gradients  
      ↓  
    optimizer update
```

So instead of updating the model after every batch, you update it after **4 batches**.

14. Why gradient accumulation?

It allows you to simulate a larger batch size without requiring enough GPU memory to hold that entire batch simultaneously.

For example:

```
Per-device batch size = 8
```

```
Gradient accumulation steps = 4
```

Effective batch size is approximately:

$$8 \times 4 = 32$$

If you're using multiple GPUs:

```
effective batch size  
=  
per-device batch size  
× gradient accumulation steps  
× number of devices
```

For example:

$$8 \times 4 \times 2 \\ = 64$$

effective batch size.

15. Very important distinction

Suppose:

```
Dataset = 1,000 examples  
Batch size = 10  
Gradient accumulation = 4
```

You have:

$$1,000 / 10 = 100 \text{ batches}$$

So:

```
100 iterations/micro-steps
```

But you don't perform 100 optimizer updates.

Instead:

```
100 / 4 = 25 optimizer steps
```

This distinction becomes very important when reading Hugging Face training logs.

16. Complete example

Let's make a realistic example.

Suppose:

```
Dataset = 10,000 examples  
Batch size = 8  
Epochs = 3  
Gradient accumulation = 4
```

Step 1 — batches per epoch

```
10,000 / 8  
= 1,250 batches
```

So approximately:

```
1,250 iterations per epoch
```

Step 2 — total batches

```
1,250 × 3  
= 3,750 batches
```

Step 3 — optimizer steps

Because:

```
gradient accumulation = 4
```

approximately:

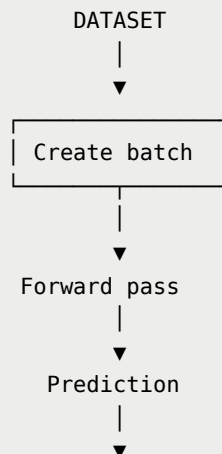
```
3,750 / 4  
= 937.5
```

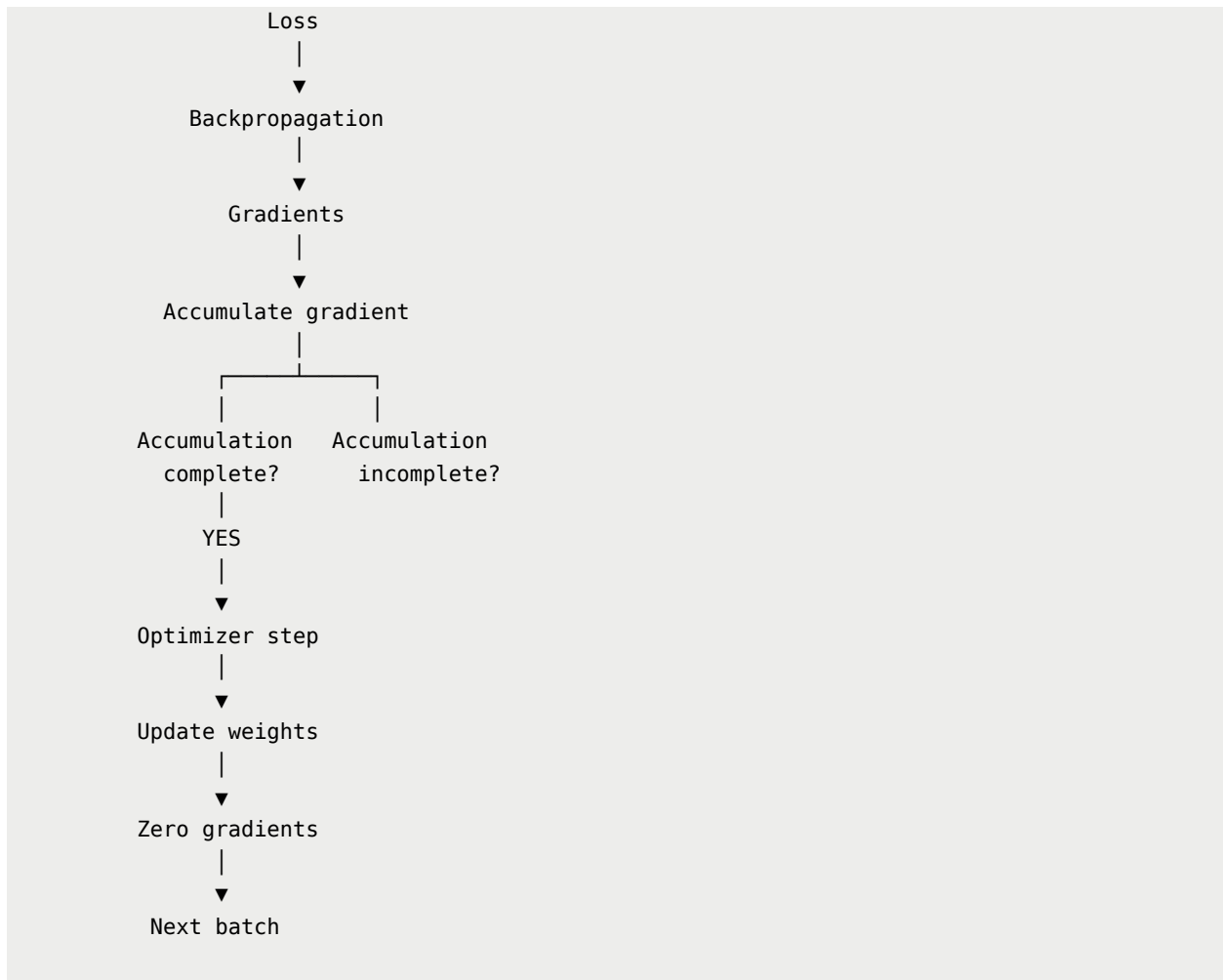
So roughly:

```
938 optimizer steps
```

assuming handling of the final incomplete accumulation follows the framework's behavior.

17. Visualizing the training loop





18. `zero_grad()`

You might see code like:

```
optimizer.zero_grad()
```

Why?

Because gradients can accumulate in memory.

Normally the pattern is:

```
optimizer.zero_grad() outputs=model(batch) loss=calculate_loss(outputs) loss.backward() optim
```

```
izer.step()
```

With gradient accumulation:

```
loss.backward()# don't update yetloss.backward()# don't update yetloss.backward()# don't  
update yetloss.backward()optimizer.step()optimizer.zero_grad()
```

So gradients from multiple mini-batches contribute to one update.

19. Micro-batch

You'll often hear:

| micro-batch

A micro-batch is basically the smaller batch processed in one forward/backward pass.

Example:

```
GPU can process 4 samples
```

So:

```
micro-batch = 4
```

Then:

```
gradient accumulation = 8
```

You process:

```
4 × 8 = 32 samples
```

before performing one optimizer update.

Thus:

```
micro-batch size = 4  
accumulation steps = 8  
effective batch size  $\approx$  32
```

20. Effective batch size

This is the batch size that the optimizer effectively "sees" per update.

For single GPU:

```
Effective batch size  
=  
batch size  $\times$  gradient accumulation
```

For distributed training:

```
Effective batch size  
=  
per-device batch size  
 $\times$  gradient accumulation  
 $\times$  number of devices
```

Example:

```
per GPU batch = 4  
GPUs = 2  
gradient accumulation = 8
```

Then:

```
4  $\times$  2  $\times$  8  
= 64
```

effective batch size.

21. Checkpoint

During training you don't want to lose everything if the process crashes.

So you save the model periodically.

For example:

```
checkpoint-100  
checkpoint-200  
checkpoint-300
```

A checkpoint can contain things like:

```
Model weights  
Optimizer state  
Learning-rate scheduler state  
Training state  
Random states
```

This allows you to resume training.

22. Evaluation

You normally have:

```
Training dataset  
Validation dataset
```

Training data:

```
used to update weights
```

Validation data:

used to evaluate the model

Example:

Train loss = 0.8
Validation loss = 1.1

If training loss keeps decreasing:

0.9 → 0.8 → 0.7 → 0.6

but validation loss starts increasing:

1.0 → 1.1 → 1.3 → 1.5

you may be **overfitting**.

23. Training loss vs validation loss

Think of it like studying.

Training questions:

Questions you've practiced

Validation questions:

New questions

If you memorize the training questions:

Training performance → excellent

```
New-question performance → poor
```

That's overfitting.

24. Warmup

When fine-tuning, you might see:

```
warmup_steps
```

Instead of immediately starting with the full learning rate:

```
LR = 0.0001
```

you gradually increase it.

For example:

```
Step 0      LR = 0
Step 100    LR = 0.00002
Step 200    LR = 0.00004
Step 300    LR = 0.00006
Step 400    LR = 0.00008
Step 500    LR = 0.00010
```

This is **learning-rate warmup**.

It can make early training more stable.

25. Learning-rate scheduler

The learning rate doesn't necessarily stay constant.

For example:

```
Start:
1e-5
```

```
Warmup:  
1e-5 → 5e-5
```

```
Then:  
5e-5 → 4e-5 → 3e-5 → 2e-5 → 1e-5
```

A **learning-rate scheduler** controls this change.

Common schedules include:

```
Linear  
Cosine  
Constant  
Cosine with warmup
```

26. Weight decay

You may also see:

```
weight_decay=0.01
```

Weight decay is a regularization technique that discourages unnecessarily large weights.

It can help reduce overfitting.

AdamW is particularly associated with decoupled weight decay.

27. Gradient clipping

Sometimes gradients become extremely large.

This can cause unstable training.

You might see:

```
max_grad_norm=1.0
```

This means gradients are clipped when their norm exceeds the specified threshold.

Conceptually:

```
Normal gradient
  ↓
  0.7
  ↓
No problem

Huge gradient
  ↓
  15
  ↓
Clip it
  ↓
More manageable update
```

This helps prevent exploding gradients.

28. Mixed precision

You'll also encounter:

```
FP32
FP16
BF16
```

These refer to numerical precision.

Traditional training:

```
FP32
```

uses 32-bit floating point.

Modern GPU training often uses:

FP16

or:

BF16

Benefits:

less GPU memory
faster computation

BF16 is particularly popular for modern LLM training/fine-tuning when hardware supports it.

29. Gradient checkpointing

Don't confuse:

checkpoint

with:

gradient checkpointing

They are different.

Normal checkpoint

Save your training state:

checkpoint-500

Gradient checkpointing

A memory-saving technique.

Instead of storing every intermediate activation during forward propagation, the model stores fewer activations and recomputes some during backward propagation.

Tradeoff:

```
Less GPU memory
      ↓
More computation
```

This is very useful when fine-tuning large LLMs.

30. Sequence length

For LLM fine-tuning you'll also see:

```
max_seq_length
```

Suppose:

```
max_seq_length = 512
```

Each training example can contain up to approximately 512 tokens.

Example:

```
"Explain machine learning..."
```

gets tokenized:

```
[token1, token2, token3, ...]
```

The model processes those tokens.

Longer sequence:

```
512 → more GPU memory  
1024 → more GPU memory  
2048 → much more GPU memory
```

This is one reason LLM fine-tuning can become memory-intensive.

31. Tokenization

Before the model sees text:

```
Text  
↓  
Tokenizer  
↓  
Tokens  
↓  
Token IDs  
↓  
Model
```

For example, conceptually:

```
"Hello world"
```

might become:

```
["Hello", " world"]
```

then:

```
[15496, 995]
```

The exact tokenization depends on the tokenizer.

32. Batch size vs sequence length

These two have a huge effect on GPU memory.

Suppose:

```
Batch size = 8  
Sequence length = 512
```

The model processes roughly:

```
8 × 512  
= 4,096 tokens
```

per micro-batch.

If you increase:

```
Batch = 16  
Sequence = 1024
```

you now have:

```
16 × 1024  
= 16,384 tokens
```

That's **4× as many tokens** per micro-batch.

So GPU memory requirements can rise substantially.

33. Fine-tuning vs pretraining

This is important.

Pretraining

You start with a model architecture and train it on enormous amounts of data.

```
Huge dataset
  ↓
Train model
  ↓
Learn general language/world patterns
```

Example:

```
Billions/trillions of tokens
```

Fine-tuning

You start with an already-trained model.

```
Pretrained LLM
  ↓
Your specialized dataset
  ↓
Fine-tuned model
```

Example:

```
Qwen
  ↓
your interview Q&A dataset
  ↓
fine-tuned Qwen
```

You're adapting an existing model rather than teaching language from scratch.

34. LoRA changes the picture

When you're fine-tuning an LLM, you may use **LoRA** rather than updating all model weights.

Normal full fine-tuning:

```
LLM
↓
Update billions of parameters
```

LoRA:

```
Original LLM weights
↓
frozen

Small LoRA adapters
↓
trainable
```

So instead of modifying billions of parameters, you train a much smaller number of additional parameters.

This dramatically reduces memory requirements.

35. Full fine-tuning vs LoRA

Full fine-tuning

```
Base model
↓
All/most weights trainable
↓
Backpropagation
↓
Optimizer
↓
Update huge number of parameters
```

LoRA

```
Base model
↓
Weights frozen
↓
```

```
LoRA adapters added
↓
Only adapters receive updates
```

The training loop still contains:

```
forward
→ loss
→ backward
→ gradient
→ optimizer
```

but far fewer parameters are updated.

36. PEFT

You'll also see:

PEFT = Parameter-Efficient Fine-Tuning

LoRA is one PEFT technique.

Other approaches exist, but conceptually:

```
Full fine-tuning
↓
Update huge number of parameters

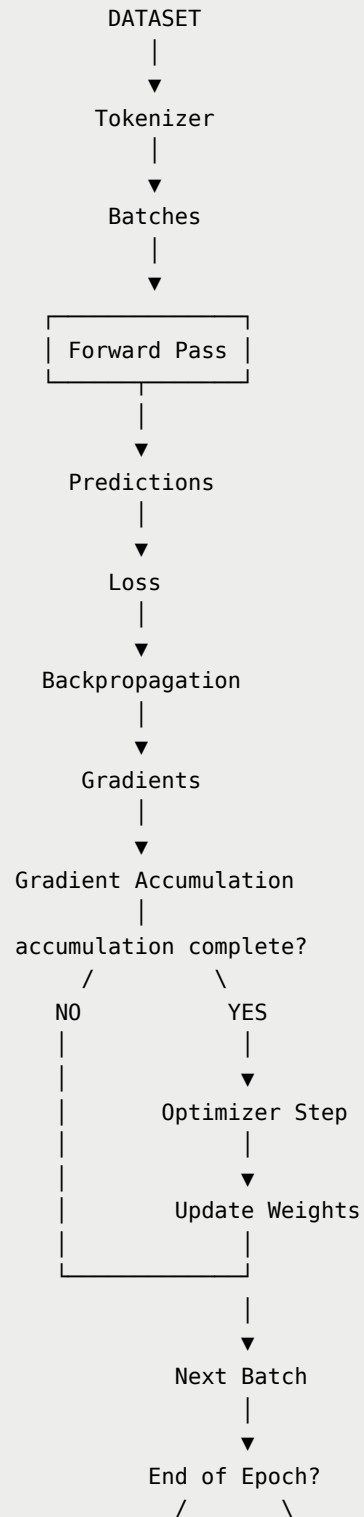
PEFT
↓
Update small subset/additional parameters
```

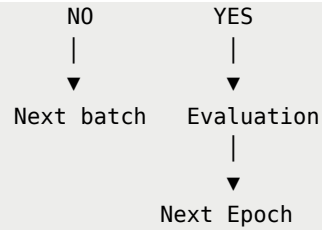
Common tools in the Hugging Face ecosystem include:

```
Transformers
PEFT
TRL
Datasets
Accelerate
```

37. The entire fine-tuning loop

Here's the most important diagram to remember:





38. One complete numerical example

Let's say you're fine-tuning a model with:

Dataset = 10,000 examples
Epochs = 3
Micro-batch size = 4
Gradient accumulation = 8
Learning rate = 2e-5

Batches per epoch

$10,000 / 4$
 $= 2,500$

So approximately:

2,500 iterations per epoch

Total iterations

$2,500 \times 3$
 $= 7,500$

Optimizer steps

Because:

```
gradient accumulation = 8
```

approximately:

```
7,500 / 8  
≈ 938 optimizer steps
```

So you could conceptually have:

```
Iteration 1  
  ↓  
Forward  
  ↓  
Loss  
  ↓  
Backward  
  ↓  
Accumulate  
  
Iteration 2  
  ↓  
Forward  
  ↓  
Loss  
  ↓  
Backward  
  ↓  
Accumulate  
  
...  
  
Iteration 8  
  ↓  
Forward  
  ↓  
Loss  
  ↓  
Backward  
  ↓  
Accumulate  
  ↓  
Optimizer Step  
  ↓  
Weights updated
```

Then:

```
Iteration 9
...
```

39. The terms you should memorize

Term	Simple meaning
Dataset	All training examples
Sample	One training example
Batch	Group of samples processed together
Micro-batch	Small batch processed in one GPU pass
Epoch	One complete pass through dataset
Iteration	Usually one batch processed
Forward pass	Model makes predictions
Loss	How wrong the prediction is
Backward pass	Calculate gradients
Gradient	Direction/magnitude of parameter adjustment
Optimizer	Uses gradients to update parameters
Optimizer step	One actual parameter update
Gradient accumulation	Combine gradients from multiple batches before updating
Learning rate	Size of parameter updates
Scheduler	Changes learning rate during training
Warmup	Gradually increases LR at beginning
Weight decay	Regularization to discourage large weights
Gradient clipping	Prevents excessively large gradients
Checkpoint	Saved training/model state
Validation	Evaluate on unseen validation data
Overfitting	Model learns training data too specifically

Term	Simple meaning
Sequence length	Maximum number of tokens processed
Mixed precision	FP16/BF16 to reduce memory/speed up training
Gradient checkpointing	Save GPU memory by recomputing activations
LoRA	Train small adapter matrices instead of all model weights
PEFT	General family of parameter-efficient fine-tuning methods

40. The single most important relationship

If you're preparing to fine-tune something like **Qwen 3 8B**, remember this chain:

```

DATASET
↓
TOKENIZATION
↓
BATCH
↓
FORWARD PASS
↓
LOSS
↓
BACKPROPAGATION
↓
GRADIENTS
↓
GRADIENT ACCUMULATION
↓
OPTIMIZER STEP
↓
WEIGHT UPDATE
↓
NEXT BATCH
↓
...
↓
END OF EPOCH
↓
VALIDATION
↓
NEXT EPOCH

```

And the three numbers you should immediately understand when you see a training configuration are:

```
batch_size
gradient_accumulation_steps
num_train_epochs
```

For example:

```
per_device_train_batch_size=4gradient_accumulation_steps=8num_train_epochs=3
```

means:

```
GPU processes 4 examples at a time
      ↓
8 such batches accumulate gradients
      ↓
1 optimizer update
      ↓
repeat
      ↓
3 complete passes over dataset
```

That's the core of the **training/fine-tuning loop**.